

Tomasulo实验

1. 实现功能与思路

1.1 bool Backend::dispatchInstruction

后端分配指令

1. 检查rob和指令对应的保留站是否已满（type为NONE时不需要保留站，且插入rob时即为ready）
2. 插入rob以及保留站（调用insertInstruction）
3. 将目的地寄存器标记为busy
4. 返回true/false

1.2 bool Backend::commitInstruction

后端提交指令（下面省略rob的弹出过程）

1. 特殊处理store指令，调用writeMemoryHierarchy
2. 特殊处理load指令：如果被invalidate，需要跳转到指令pc，并flush；否则写寄存器
3. 特殊处理B以及J指令，
 1. 更新bpu
 2. J指令写寄存器
 3. 如果mispredict，跳转到正确的地址，flush
4. 若为EXIT指令，返回值是true
5. 剩下的指令，根据需要写寄存器

1.3 void ReservationStation::insertInstruction

向保留站插入指令

1. 遍历找到第一个空slot，否则返回
2. 置为busy，修改相应inst等等字段
3. 设置readport
 1. 检查对应的寄存器是否busy，空闲则直接读，无需等待唤醒
 2. 不空闲则检查rob中是否准备好，并根据情况置或不置waitForWakeup字段，设置其他字段

1.4 void ReservationStation::wakeup

唤醒保留站中等待当前rob结果的指令

1. 遍历保留站中的项，选择其中busy的项
2. 比较正在等待的readPort的robIdx是否与该即将写的rob项的index相同，相同则唤醒readPort

1.5 bool ReservationStation::canIssue

检查保留站中是否有准备好可以发射的指令。这里对所有保留站实现了实验指导书推荐的压缩队列

而保留站是一个顺序压入，乱序弹出的结构。由于 Store 指令需要由保留站保证执行顺序，因此建议在维护保留站信息时，使用压缩队列的方式进行维护。即永远保证保留站内的项目有序，且集中在小标号一侧。下面是一个例子：

```
| a | b | c | - | 压入指令 a, b, c  
| a | c | - | - | 弹出 b  
| a | c | d | e | 压入指令 d, e  
| a | c | e | - | 弹出 d  
| c | e | - | - | 弹出 a
```

1. 如果buffer的第一项非busy，表示整个队列为空，直接返回
2. 特殊判断store指令，只需要观察第一项是否准备好
3. 剩下的指令按序遍历所有非busy的项，如果有准备好的项（两个readPort都不等待）则返回true，否则返回false

1.6 IssueSlot ReservationStation::issue

返回一条准备好发射的指令，并从保留站中弹出

1. 特殊处理store指令，由于需要按顺序，直接返回第一项
2. 对其他剩下指令，按序遍历busy的项，找到第一个准备好的项并返回
3. 注意在返回结果前其实是将结果保存起来，先将结果后面的busy项全部前移一位，再返回（为了维护压缩队列）

1.7 std::optional< unsigned>StoreBuffer::query

查询storebuffer中在输入的robIdx之前（时间顺序）的最近的、具有相同地址的项

1. 从后先前（实际上是从pushptr到popptr）遍历整个buffer
2. 找到第一项满足地址相同、时间上在输入的robIdx之前的项（时间顺序的判断为 `(robPopPtr <= buffer[i].robIdx && buffer[i].robIdx < robIdx) || (robPopPtr <= buffer[i].robIdx && robIdx < robPopPtr) || (buffer[i].robIdx < robIdx && robIdx < robPopPtr)`）并返回或者返回std::nullopt

1.8 void LoadBuffer::check

将loadbuffer中在输入的robIdx之后（时间顺序）且具有相同地址的所有项标记为invalidate

1. 遍历所有valid项
2. 判断地址相同、时间顺序 `(robPopPtr <= buffer[i].robIdx && buffer[i].robIdx < robIdx) || (robPopPtr <= buffer[i].robIdx && robIdx < robPopPtr) || (buffer[i].robIdx < robIdx && robIdx < robPopPtr)` 并标记

1.9 选做BranchPredictBundle

FrontendWithPredict::bpuFrontendUpdate

前端返回分支预测结果。实现的是直接映射，有1024个表项的两位饱和的BTB。使用pc的[11:2]作为index。

1. 与操作计算index，获取表项
2. 如果valid且pc值对应，且计数器大于1，则预测跳转

1.10 选做unsigned FrontendWithPredict::calculateNextPC

计算pc与上面的逻辑相同，如果预测跳转则返回BTB表项中的target，否则直接pc+4

1.11 选做void FrontendWithPredict::bpuBackendUpdate

后端更新

1. 若为valid表项且pc与输入相同，则更新计数器
 1. 若为branchTaken，则计数器更新为1（原计数器为0）或者3（其他）
 2. 否则，将计数器更新为2（原计数器为3）或者0
2. 否则计数器置1,记录目标地址，置valid，记录pc

2. git commit hash

f6a445fc6c9e8d7ee48e2133c744597359e12fbb